

求职主线：**主后端 + Python AI 自动化 + 强前端全栈交付**。我已经做出上线系统的业务后端能力证明，但不再作为求职主标题。我的核心价值是把后台业务、认证权限、队列、实时通信、AI 工具链、Web / 移动端 / 桌面端前端和部署链路收束成真实可运行的系统。

基本信息

姓名：方中杰

电话：15555339309

地点：武汉 / 全国 / 远程均可

技术社区：[Linux.do 三级用户]

学历：本科 · 安徽信息工程学院 · 软件工程 · 2026届

邮箱：2809726152@qq.com

期望薪资：8K 起

个人定位

我现在的技术定位不是“全栈”，也不是“只会前端”。更准确地说，我是一个以**后端和 Python AI 自动化为主线、前端能力很强的全栈工程师**。

后端方向，我已经不是停留在语法或 CRUD Demo，而是在用 Gin / GORM / Redis 推进一个企业级 Admin 后端核心迁移：认证会话、RBAC、用户管理、角色授权、操作日志、队列监控、系统设置、上传配置、COS 上传 token、WebSocket baseline 和 smoke 验证都已经形成代码和测试闭环。

Python 方向，我把它放在 AI 应用、数据处理、自动化脚本和内容流水线里，而不是硬把 Python 包装成普通 CRUD 后端。电商商品采集、图片/OCR、AI 口播生成、TTS、SRT、批量文件处理、接口调试和 AI 工具链验证，都是 Python 更适合发力的位置。

前端方向，我的能力不能被弱化。我不仅能写后台页面，还能做工程基础设施、复杂权限菜单、动态路由、状态管理、跨端移动端、桌面端和 UI 生成工作流。Vue、React、TypeScript、uni-app、Vant、Element Plus、Ant Design、Electron、Tauri、Capacitor 这类能力，是我能把后端和 AI 能力真正交付给用户的关键。

核心能力

Go 后端 / Admin Core / 系统重构

- 使用 Go / Gin / GORM / MySQL / Redis / go-redis / slog / context 构建 Admin modular monolith，顶层按 `cmd -> bootstrap -> server -> module -> platform` 装配，模块内部按 `route -> handler -> service -> repository -> model` 收口。
- 已推进企业级 Admin Go 后端核心基建：`auth`、`session`、`authplatform`、`captcha`、`user`、`permission`、`role`、`operationlog`、`queuemonitor`、`systemsetting`、`uploadconfig`、`uploadtoken`、`realtime` 等模块。
- 熟悉认证会话链路：Access / Refresh Token、Token Hash + Pepper、Redis token cache、MySQL session fallback、平台策略、设备/IP 绑定、单端登录、refresh rotation、logout revoke。
- 熟悉 RBAC Admin 权限体系：单角色模型、`DIR / PAGE / BUTTON` 权限类型、动态菜单、动态路由、按钮权限码、角色授权、缓存失效、fail-closed 接口权限检查。
- 能设计明确 middleware 顺序：`Recovery -> RequestID -> AccessLog -> CORS -> AuthToken -> PermissionCheck -> OperationLog -> Handler`，认证、权限和审计互不污染。
- 能处理 Go 后端运行边界：readiness、graceful shutdown、统一 response / app error、显式 route metadata、table-driven tests、smoke scripts、队列 worker、WebSocket connection manager。

Python / AI 应用 / 自动化流水线

- 能用 Python 承接 AI 应用中的采集、清洗、批处理、接口回放、文件处理、素材处理、模型调用验证、评估脚本和自动化任务。
- 有电商商品数据、图片/OCR、AI 卖点与口播生成、TTS 合成、SRT 字幕下载这类内容生产流水线经验，理解 AI 应用不是一个“调用模型按钮”，而是一条可排队、可追踪、可重试、可审核的任务链。
- 能把 Python 自动化与 Web 后台结合：前端负责操作入口和审核体验，Go/PHP 后端负责状态、权限、队列和持久化，Python/脚本层负责重复处理、数据处理和 AI 工具链。
- 理解 Python 在 AI 生态里的优势：LLM/RAG、OCR、TTS、embedding、数据处理、脚本自动化、评估与批量任务；不把 Python 误用成所有系统的唯一后端。

AI Agent / LLM 工程化

- 能把 LLM 调用落成工程系统：模型接入、Agent 配置、Prompt 管理、工具调用、流式输出、运行审计、超时取消和错误暴露。
- 设计过 Agent、Model、Tool、Prompt、Conversation、Message、Run、Run Step 等运行模型，把一次 AI 调用从“黑盒请求”拆成可观测状态机。
- 熟悉 OpenAI-compatible 接口、SSE / WebSocket 实时输出、历史消息拼装、工具执行记录、结果截断、失败恢复和取消机制。
- 关注 Tool Calling 安全边界：HTTPS 白名单、SSRF 防护、只读 SQL、写操作拦截、自动 LIMIT、敏感结果控制。

前端 / 移动端 / 桌面端工程

- 熟悉 Vue 3、React、TypeScript、Vite、Element Plus、Ant Design、Vant、Pinia、Zustand、Vue Router、React Router、Vue I18n、TanStack React Query、Tailwind CSS。
- 具备后台前端工程化能力：统一请求封装、ApiEnvelope 解包、401 刷新队列、动态路由、按钮权限、权限快照、CRUD Hook、表格列配置、搜索表单、Dialog 体系和 i18n 文案边界。
- 有跨端移动端能力：uni-app、H5、Android、iOS、微信小程序、鸿蒙配置、移动推送、腾讯 IM / TRTC、Vant 问诊 H5；也能处理 Capacitor 这类 Web 技术栈到移动端壳层的集成思路。
- 有桌面端能力：Electron / Tauri，能处理 Web/Desktop 双运行链路、preload / bridge、IPC、本地后端 ready、窗口能力、安装包构建、更新清单和 CSP 安全策略。
- 有 UI 生成和前端质量治理经验：能把 Figma Make / AI 生成代码收口到项目组件体系，不让生成代码污染业务层；也能设计本地 UIUX patch compiler 工作流。

PHP / 业务系统 / 存量交付

- 熟悉 PHP 8.1+、Webman / Workerman、Laravel、Eloquent、MySQL、Redis、Redis Queue、GatewayWorker、Crontab、PHPUnit。
- PHP 对我不是包装词，而是已上线业务系统的证据：认证权限、AI Agent、SSE、WebSocket、支付钱包、订单履约、上传存储、系统通知、导出任务、日志审计和线上部署都做过。
- 但求职表达上，PHP 不再抢主线；它作为存量业务系统维护、迁移事实提取和并行重构能力存在，主线转向 Go 后端、Python AI 自动化和强前端全栈。

部署 / 交付 / 工程纪律

- 独立完成域名、HTTPS、Nginx 反代、Webman 多端口服务、Go API、MySQL、Redis、COS 静态资源、桌面端更新清单配置。
- 能区分 API、SSE、WebSocket、队列 worker、桌面端本地能力、移动端跨端运行和外部渠道回调的运行形态，并按协议特性做隔离。
- 关注契约测试、接口边界和工程约束，不靠前端空数组、空对象、any、静默 catch 或后端兜底字段掩盖协议错误。

工作经历

武汉晶音科技有限公司 · 全栈开发

时间：2025.6.11 - 2025.9.01

- 参与半导体行业交流平台微信小程序开发，基于 Java 与 Vue 2 完成页面迭代、业务逻辑优化及接口开发，持续打磨交互细节并完善核心功能。
- 参与版本测试、问题修复与上线发布，基于 Git 进行协同开发与版本管理，并完成站点部署流程及测试脚本编写。

小药药医药科技有限公司 · 前端开发

时间：2025.9.16 - 至今

- 从 0 搭建公司 SaaS 商家端前端工程，覆盖 Web 独立运行与 Electron Desktop 打包运行两条链路。
- 负责登录、租户/门店选择、总部/门店工作区切换、权限菜单、会话恢复、统一请求、状态管理和 CRUD 基础设施等核心工程能力。
- 参与荷叶问诊后台、移动端 APP、问诊内核 H5 三端迭代，覆盖远程审方、视频问诊、合理用药审查、长处方/慢病规则、移动推送与跨端跳转等医疗问诊核心链路。
- 在 Figma Make 生成代码质量不稳定、UI 基础组件边界不清的约束下，逐步收口页面结构、组件边界和交互规范，避免设计稿代码直接污染业务层。

项目经历

Admin Go 主后端 Core Foundation 迁移

- **角色**：个人项目 / Go 主后端架构与核心实现 / 既有 PHP Admin 系统并行重构
- **项目路径**：[E:\admin_go\admin_back_go](#)
- **技术栈**：Go、Gin、GORM、MySQL、Redis / go-redis、Asynq、gocron、gorilla/websocket、slog、context、RESTful API、RBAC、Token Session、COS STS、Table-driven Tests。
- **相关复盘**：[Go Admin Core Foundation：从 PHP 迁移到 Gin Modular Monolith](#)

项目概述

该项目是我围绕既有企业级 Admin 系统推进的 Go 主后端重构。它不是重新写一个玩具 CRUD，而是在不破坏现有前端、登录、菜单、按钮权限和业务使用路径的前提下，把旧 PHP 系统里的认证、会话、RBAC、用户管理、日志、队列、上传和实时通信边界逐步迁到 Go。

当前 Go 后端已经进入 **Admin core foundation** 阶段：不只是有骨架，而是已经有认证会话、RBAC read/write path、用户管理、个人资料、账号安全、系统日志、操作日志、队列监控、系统设置、上传配置、COS 上传 token、WebSocket baseline、basic/full smoke 和单元测试体系。

核心工作

- 采用 **Gin modular monolith**，固定 `cmd -> bootstrap -> server -> module -> platform` 和 `route -> handler -> service -> repository -> model`，拒绝 Java 味 `ServiceImpl`、无意义 interface、handler 查库和 service 依赖 `gin.Context`。
- 实现认证会话核心链路：登录配置、滑块验证码、密码/验证码登录、Access / Refresh Token、Token Hash + Pepper、Redis token cache、MySQL session fallback、refresh、logout、平台策略、设备/IP 绑定、单端登录和登录日志 task。
- 迁移 RBAC 核心：`Users/init` legacy adapter、`GET /api/admin/v1/users/init`、动态 router、permissions、buttonCodes、`DIR / PAGE / BUTTON` 权限类型、角色授权、权限树、按钮缓存、权限变更后的用户授权缓存失效。
- 建立显式中间件边界：`AuthToken` 只做认证，`PermissionCheck` 只按 route metadata 做 fail-closed 权限检查，`OperationLog` 只记录显式配置的操作审计，不靠反射、注解或 handler 名称猜测。
- 迁移基础管理模块：用户管理 page-init/list/edit/status/delete/batch update、个人资料、账号安全、系统设置、系统日志、操作日志、认证平台管理、权限管理、角色管理。
- 建立后台任务边界：`cmd/admin-api` 只处理 HTTP，`cmd/admin-worker` 负责队列消费和 scheduler；使用 Asynq 封装 critical/default/low lane，`scheduler` 只投递 task，不直接执行业务。
- 实现上传配置与运行时 token：upload drivers/rules/settings REST 管理、VAULT_KEY secretbox、COS-first STS 临时凭证签发、服务端生成 object key、folder/ext/size 校验，不把 OSS runtime 和无主上传场景硬塞进默认链路。
- 实现 Realtime / WebSocket baseline：认证后的 `/api/admin/v1/realtime/ws`、path-scoped browser cookie auth、local connection manager、bounded send queue、read/write pump、connected event、ping/pong、topic subscribe 白名单骨架、local/no-op Publisher 边界。
- 建立验证门禁：当前仓库约 229 个 Go 文件、70 个测试文件、365 个测试函数；`go test ./...`、`go vet -p=1 ./...`、`git diff --check` 已验证通过；basic/full smoke 覆盖登录、验证码、RBAC、用户管理、队列、上传、日志和 WebSocket 基础链路。

求职价值

这个项目是我 Go 后端能力的核心证明：我不是只会 Gin CRUD，而是能把一个已有复杂后台系统拆出真实边界，再用 Go 重建认证、会话、RBAC、审计、队列、上传、WebSocket 和测试体系。它同时说明我懂迁移节奏：旧 PHP 提供业务事实，Go 负责新主后端，前端路径不能被破坏，Python 后续承接 AI 和数据处理 sidecar。

Python + AI 电商内容自动化流水线

- **角色**：个人项目 / Python AI 自动化 / 商品 AI 工作台能力
- **技术栈**：Python 自动化脚本、Chrome Extension、OCR、AI Agent、TTS、Redis Queue、PHP / Webman、MySQL、COS、SRT。

项目概述

该项目围绕电商商品构建 **商品采集 -> 图片/OCR -> AI 卖点与口播生成 -> TTS 合成 -> SRT 下载** 的内容生产流水线。它的价值不是“调一个模型接口”，而是把运营里的重复劳动拆成可采集、可清洗、可排队、可追踪、可重试、可审核的任务链。

核心工作

- 浏览器插件采集商品标题、价格、销量、品牌、店铺、规格、描述、评论、详情图等结构化信息。

- Python / 脚本层辅助批量数据清洗、图片/文件处理、接口调试、AI 工具链验证和重复任务自动化。
- 后端承接商品入库、图片选择、OCR 识别、Agent 生成卖点/口播词、TTS 合成和字幕文件下载。
- 使用队列承载 OCR、AI、TTS 等耗时任务，为每个阶段设计状态流转，避免任务失败后只留下模糊的“生成失败”。
- 该项目适合作为 Python / AI 应用方向证明：Python 负责 AI 工作流和自动化，Web 后台负责状态、权限和人工审核，形成真实业务闭环。

SaaS 商家端 Web / Desktop 一体化前端

- **角色：**公司项目 / 前端架构与核心开发 / 从 0 搭建
- **技术栈：**React 19、TypeScript、Vite、Electron、Ant Design、TanStack React Query、Zustand、React Router、Axios、Zod、Tailwind CSS、Electron Builder。

项目概述

该项目是面向药店/商家的 SaaS 管理端，既支持浏览器 Web 运行，也支持 Electron 桌面端本地运行和打包发布。我负责从工程初始化到核心链路落地：运行时识别、接口基址配置、登录恢复、门店/总部工作区、权限菜单、统一请求、状态管理和通用 CRUD 页面能力。

核心工作

- 搭建 Web / Desktop 双运行链路：Web 端使用 Vite 独立构建，桌面端通过 Electron 承载本地运行、后端 ready bridge、窗口能力和安装包构建。
- 设计运行时初始化与接口客户端配置，区分 Web、Desktop、本地后端、业务 API 等不同基础地址，避免页面代码到处判断运行环境。
- 落地登录、Token 恢复、门店选择/申请、总部/门店工作区切换、权限菜单和动态路由入口，让用户会话和业务工作区状态可恢复、可切换。
- 封装 Zustand session/users 状态、Axios 请求客户端、统一错误处理、权限快照和 i18n 文案边界，沉淀 Dialog、Search、Table、Column Settings、CRUD Hook 等可复用能力。
- 坚持强契约开发：前端不靠猜字段、空兜底和静默 catch 掩盖接口问题，优先让协议错误暴露出来，再按真实契约修正。

荷叶问诊医药 SaaS 三端协同系统

- **角色：**公司项目 / 核心前端开发
- **技术栈：**Vue 3、TypeScript、Vite、Element Plus、Vant、uni-app、Pinia、Axios / luch-request、腾讯云 IM / TRTC、阿里云移动推送、Aegis。

项目概述

该项目覆盖 PC 管理后台、跨端移动 APP 和问诊内核 H5。核心业务不是普通页面 CRUD，而是围绕互联网医院与药店问诊场景，把患者、医生、药师、商家、处方、审方、视频通话、移动推送和合规规则串成可用链路。

核心亮点

- 后台端承接远程审方、药师工作台、处方记录、问诊数据、GSP 商品资料和合理用药规则配置，处理动态路由、租户 `tenant-id`、Token 刷新队列和全局错误提示。
- 移动端基于 uni-app 支持 H5、Android、iOS、微信小程序和鸿蒙配置，接入阿里云推送、腾讯 IM / TRTC，处理处方待审、视频来电、订单通知、权限申请和跨端页面跳转。
- 问诊内核 H5 使用 Vue 3 + Vant 承载患者问诊、医生接诊、药师审方、第三方问诊流转、商品提交、套餐购买和消息中心等移动端业务。
- 在第三方问诊链路中梳理详情拉取、合理用药审查、药品/诊断归一化、审查弹窗拦截和后续流转，避免把医疗规则判断散落在页面事件里。
- 排查过长处方/慢病仍提示超量的问题，定位到“慢病病情选择”和“后台慢病目录药品配置”是两层不同规则，问题根因不在前端状态，而在规则目录匹配。

AI Make 本地 UI/UX Patch Compiler

- **角色：**个人项目 / 产品设计与独立开发 / Codex Skill + npm CLI
- **技术栈：**Node.js、JavaScript ESM、Codex Skill、Prompt Engineering、React、TypeScript、Tailwind CSS、Multi-Agent Workflow。

项目概述

AI Make 是我围绕 Figma Make 工作流沉淀的本地开发者 UI 生成 Skill。它不是网页 IDE，也不直接调用模型 API，而是把一句 UI 需求编译成 **visual brief**、**page composition blueprint**、**ui-spec**、**prompt-pack**、**agent handoff**、**任务拆分**和 **review gates**，再交给 Codex / Claude Code / Cursor 等本地编码 Agent 在现有项目里生成可审查、可验证、可合并的前端 patch。

核心工作

- 设计从自然语言 UI 需求到本地代码 patch 的编译链路：先确定视觉方向和首屏结构，再生成规格、提示词包、Agent 交接文档和验证要求。
- 实现 `ai-make` CLI 与 Codex Skill 入口，支持读取目标项目上下文、生成运行目录、拆分任务、输出单任务 Agent prompt，并记录本轮 patch 的 review gate。
- 把视觉质量放在代码质量之前：通过 `visual-brief` 和 `page composition blueprint` 约束首屏框架、hero、指标节奏、证据区、行动区和细节层，避免生成“能跑但像模板”的页面。
- 强约束生成边界：不猜后端字段、不添加 fallback 字段、不绕过项目架构、不擅自引入新 UI 库、不生成巨大单文件页面。

技术文章 / 证明材料

- Go Admin Core Foundation: 从 PHP 迁移到 Gin Modular Monolith
- Go 语言基本学习路线: 从变量到项目入门
- 从调 API 到 Agent 工程化: 把 AI 能力做成可治理系统
- Agent 工程学习路线: 从 LLM 到可上线智能体系统
- 电商 AI 口播生成系统: OCR、Agent、TTS 与队列闭环
- Webman 分层架构: Controller 到 Model 的边界治理
- SSE 流式对话系统: AI 实时输出的生产级实现
- 医疗问诊 SaaS 三端协同: 后台、移动端与问诊内核怎么串起来

教育经历

安徽信息工程学院 · 软件工程 · 本科

毕业时间: 2026.06

我的优势

- Go 不是贴标签**: 我已经用 Go 推进 Admin core foundation，覆盖认证会话、RBAC、用户管理、队列、上传、WebSocket、日志和测试门禁，不是只写过 Gin CRUD。
- Python 有真实位置**: Python 负责 AI 应用、数据处理、自动化脚本和内容流水线，不硬塞成普通 CRUD 后端。
- 前端能力很强**: 能做 React / Vue 后台工程、uni-app 移动端、Vant H5、Electron / Tauri 桌面端、权限菜单、状态管理、跨端运行和 AI UI 生成 workflow。
- PHP 是上线证明，不是主线包袱**: PHP / Webman 系统证明我能交付复杂业务，但求职主线已经转向 Go 后端、Python AI 自动化和强前端全栈。